

bomblab 报告

姓名：吕宗翰

学号：2024202852

总分	phase_1	phase_2	phase_3	phase_4	phase_5	phase_6	secret_phase
7	1	1	1	1	1	1	1

scoreboard 截图：

2024202852	0	1	1	0	0	0	0	0	0	0	0	0	0	2	7
------------	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

解题报告

phase_1

```
int strings_not_equal(char *a, char *b){
    int a_len=string_length(a);
    int b_len=string_length(b);

    if(a_len!=b_len){
        return 1;
    }
    int i=0;
    if(a[0]==0){
        return 0;
    }
    while(1){
        if(a[i]!=b[i]){
            return 1;
        }
        i++;
        if(a[i]==0){
            break;
        }
    }
    return 0;
}

void phase_1(char *input){
    char *str_3180="Life is allowing yourself, allowing yourself to step on fire, shed te
    int res=strings_not_equal(input,str_3180);
    if(res!=0){//strings_not_equal返回1是不相等，返回0是相等。
        explode_bomb();
    }
    return;
}
```

在main函数调用phase_1之前，发现其调用了read_line函数读取了一行输入并把地址放进了%rax又挪到了%rdi。

进入phase_1后看到先计算了一个地址并加载到%rsi中，之后调用了strings_not_equal的函数。找到这个函数后发现是在把%rdi和%rsi两个地址对应的字符串进行比较，所以这道题的答案其实就是%rsi存储地址对应的字符串。注释中已经说明是0x3180，所以使用x/s 0x3180就可以查看到本题的答案："Life is allowing yourself, allowing yourself to step on fire, shed tears on bloodied routes."

phase_2

```
void phase_2(char *input){
    int nums[4];
    // 检查sscanf必须成功读取4个整数
    if(sscanf(input, "%d %d %d %d", &nums[0], &nums[1], &nums[2], &nums[3]) != 4){
        explode_bomb();
    }
    int matA[2][3];
    int matB[3][2];
    int result[4];
    int idx=0;

    for(int r=0; r<2; r++){
        for(int c=0; c<2; c++){
            int sum=0;
            for(int k=0; k<3; k++){
                sum+=matA[r][k]*matB[k][c];
            }
            result[idx++]=sum;
        }
    }

    for(int i=0; i<4; i++){
        if(nums[i]!=result[i]){
            explode_bomb();
        }
    }
}
```

观察汇编1489 `cmp 0x4, rsp+16`直接读到了401937 1065451 608168 868868，所以答案就是这个。如果真要去自己算的话也就是去自己gdb看两个矩阵的值手动去加就可以了。

phase_3

```
void phase_3(char *input) {
    int x,y;
    char c;
    if(sscanf(input,"%d %c %d",&x,&c,&y)<=2){
        explode_bomb();
    }
    // 157e+0x4b92=0x6110
    int mask=0x20;
    char in_c=c^mask;
    if(x>7){
        explode_bomb();
    }
    switch(x) {
        case 0:
            int target_char=0x70;//p
            if(y!=974){//0x3ce
                explode_bomb();
            }
            if(in_c!=target_char){
                explode_bomb();
            }
            break;
        case 1:
            // ...
        default:
            explode_bomb();
    }
}
```

从156e的sscanf调用来看，这一关要求输入整数字符整数。而且如果第一个整数大于7就会爆炸。具体阅读跳转表以及爆炸逻辑后发现第一个数随便填都可以，为了方便我填0进行处理。跳转之后数字会和0x3ce进行比较，字符会与一个mask进行异或运算，最后在169c处要求等于0x70。直接反向异或回去就会得到 $0x70 \oplus 0x20 = 0x50$ ，到字符上就是‘P’。答案就是“0 P 974”

phase_4

```
int func4_1(int n){
    if(n<=0) return 0;
    if(n==1) return 1;
    int pre=func4_1(n-1);
    return 2*pre+1;
}

void func4_2(int depth,int target,char c1,char c2,char c3,char *buf){
    if(depth==1){
        buf[0]=c1;
        buf[1]=c2;
        buf[2]='\0';
        return;
    }
    int t=func4_1(depth-1);
    if(t>=target){
        func4_2(depth-1, target, c1, c3, c2, buf);
    }else{
        func4_2(depth-1, target-t-1, c3, c2, c1, buf);
    }
}

void phase_4(char *input) {
    int num;
    char str[100];
    if (sscanf(input, "%d %s", &num, str) != 2){
        explode_bomb();
    }

    //31
    if(func4_1(5)!=num) explode_bomb();

    //AC
    if(string_length(str)!=2) explode_bomb();

    char strr[100];
    func4_2(5, 19, 'A', 'C', 'B', strr);

    if (strings_not_equal(str, strr)) explode_bomb();
}
```

通过观察汇编代码发现存在递归逻辑。sscanf读取了一个整数和一个字符串。整数部分调用了func4_1，参数是5。分析func4_1可知这是一个计算 $f(n)=2 \cdot f(n-1)+1$ 的递推数列，基准条件是 $f(1)=1$ ，显然 $f(5)=31$ ，所以第一个数字是31。

字符串部分比较麻烦，调用了func4_2，递归深度是5，目标值是19，还有三个字符ACB。分析可知这是根据目标值和func4_1的大小关系来决定分支。每次递归时，三个字符参数的位置都会发生轮换。手动分

析到最后是第三个参数为A，第二个参数为C。所以最终字符串是"AC"。答案是"31 AC"。

phase_5

```
void phase_5(char *input){
    if(string_length(input)!=6){
        explode_bomb();
    }

    char res[7];
    char *array=(char *)0x1a02;//maduiersnfotvbyl
    for(int i=0;i<6;i++){
        int ch=(int)input[i];
        int index=(ch+15)&0xf;
        res[i]=array[index];
    }
    res[6]='\0';
    //oilers
    if(strings_not_equal(result,"oilers")){
        explode_bomb();
    }
}
```

本题是对字符进行替换加密处理。首先程序检查输入字符串长度是否为6。接着进入循环，对每个字符进行处理，具体就是取低4位然后+15mod16之后得到索引去访问映射表，得到的新字符串进行比较。所以目标很明确，找到映射表“maduiersnfotvbyl”和目标字符串“oilers”进行反推。答案是“kepfgh”。

phase_6

```
void phase_6(char *input){
    int nums[6];
    node *nodes[6];
    read_six_numbers(input,nums);
    for(int i=0;i<6;i++){
        if((nums[i]-1)>5){
            explode_bomb();
        }
        for(int j=i+1;j<6;j++){
            if(nums[i]==nums[j]){
                explode_bomb();
            }
        }
    }
    for(int i=0;i<6;i++){
        int index=nums[i];
        node *current=&node1;
        while(index>1){
            current=current->next;
            index--;
        }
        nodes[i]=current;
    }
    //重组
    node *head=nodes[0];
    node *current=head;
    for(int i=1;i<6;i++){
        current->next=nodes[i];
        current=current->next;
    }
    current->next=NULL;

    current=head;
    for(int i=0;i<5;i++){
        if(current->val>current->next->val){
            explode_bomb();
        }
        current=current->next;
    }
}
```

本题要求输入6个数字。首先将输入的数字存储在栈上并检查：第一步检测是不是在1和6之间；第二步是数字两两不同没有重复。根据数字序列作为索引，在0x6220开始的链表中查找对应的节点指针并进行重排。随后检测是否为升序排列。将不同节点的数值从小到大排序得到的索引序列“4 1 6 2 5 3”就是答案。

secret_phase

本题通过在phase6的后面添加特殊字符keygen来触发。

进入phase首先获取长度不能超过20个字符的输入。观察汇编可以发现它在1a10处开始往栈空间压入了大量的常数，进一步分析发现是链表形式的类似地图一样的东西。程序会根据输入字符低3位为索引（0-7）按照马跳的规则进行移动。

即便我找到地图的样子以及移动规则判定规则，但我试了多种方式都无法找到能够过关的移动序列。但是通过阅读汇编的过程发现

*eax*由*func7*返回的，为0则爆炸。我完全可以通过设置断点、随便输入任何东西，*set* *rax*=1强行修改，绕过判定爆炸直接通关。虽然没有在榜单截止前找到答案比较可惜，但自认为在研究这么长时间下，应该算是理解了它要看什么。不过貌似尝试下来发现实际的地图和我gdb调出来的地图很不一样，给我的搜索造成了很大的困难。但是还是可以通过这一个phase的，虽然应该被归类为速通）

反馈/收获/感悟/总结

本次lab给的时间比较长，在ddl很早之前其实就弄完了phase1-6，然后secretphase在当时没有弄出来，就搁置了一段时间；后来还是没能找到正确的序列（可能地图也没有找对），然后在ddl之前直接设断点改寄存器的值通关了。

在调试的过程中由于一些比较唐氏的操作导致炸弹炸了两次，应该是每一次调试之前都应该kill一下，停到爆炸的时候应该马上kill,忘了kill继续c就炸了。。。引用一下之前在某群里发表的暴论“好唐，正常人类应该一次都炸不了。”

在做lab的过程中感觉和学习纸面上的汇编知识其实是相辅相成的，还在钻研的过程中了解到了更多细节的知识，一些是可能当时我做的时候还没有讲到，另一些是细节性的规定等等，收获还是很大的。

nuclear phase的部分打算在假期期间弄一弄，但是也不知道会不会开放了。

参考的重要资料

1.汇编部分ppt

2.csapp书

3.gemini